课程笔记

CS336 第 7 讲：并行训练基础与 ZeRO / Pipeline / Tensor Parallel

Codex 基于 Stanford CS336 2025 第 7 讲视频、字幕与官方课程材料重写

2026-04-07

视频作者/频道：Stanford Online
发布日期：2025 年 05 月 08 日
视频时长：1:24:42
视频链接：`https://www.youtube.com/watch?v=l1RJcDjzK8M`

目录

1 为什么并行训练会成为大模型时代的默认前提 **2**
1.1 本章小结 2

2 Collective communication：为什么分布式训练先从网络原语开始 **2**
2.1 all-reduce、all-gather、broadcast、reduce-scatter 是什么 2
2.2 TPU 与 GPU 在通信上的差异为什么重要 3
2.3 本章小结 3

3 Data Parallel 与 ZeRO：最自然也最容易撞墙的并行起点 **3**
3.1 Naive data parallel 的优点和硬伤 3
3.2 ZeRO stage 1 / 2 / 3 在本质上分别解决什么 4
3.3 为什么课程说 ZeRO stage 3 虽好但并不万能 5
3.4 本章小结 5

4 Model Parallel：为什么仅靠数据并行还不够 **5**
4.1 Pipeline parallel 想解决什么 5
4.2 为什么 pipeline 常被放在慢链路上 6
4.3 Tensor parallel 为什么看起来更“优雅” 6
4.4 为什么 sequence parallel 会自然冒出来 6
4.5 本章小结 6

5 为什么真实训练往往要把多种并行同时用上 **6**
5.1 单一并行策略很少独自解决全部问题 6
5.2 课程真正想教的是“哪种通信花在什么地方” 7
5.3 本章小结 7

6 总结与延伸 **7**

1 为什么并行训练会成为大模型时代的默认前提

第 7 讲一上来就把单位从“单张 GPU”提升到了“数据中心”。如果说前几讲还主要在研究单卡上的资源账本，那么这一讲等于承认一个现实：很多现代模型根本放不进单卡，也很难在单卡上用合理 wall-clock time 训练完。因此，并行训练不是附加优化，而是规模化训练的起点条件。

课程把目标说得很清楚：希望多 GPU / 多机器系统至少同时做到两件事，一是内存近似线性扩展，二是计算近似线性扩展。也就是说，GPU 数翻倍后，你不只是想“勉强多放一点参数”，而是最好也真能更快训起来。

第 7 讲的核心任务

把“多卡训练”从模糊印象拆成一套清晰的并行原语：collectives、data parallel、ZeRO、pipeline parallel、tensor parallel 和 sequence parallel。

1.1 本章小结

第 7 讲并不只是教你一些分布式名词，它是在把第 2 讲的资源会计扩展到多卡环境：显存怎么拆、梯度怎么同步、参数怎么汇聚、通信怎么隐藏。

2 Collective communication：为什么分布式训练先从网络原语开始

2.1 all-reduce、all-gather、broadcast、reduce-scatter 是什么

课程先故意放慢节奏，讲 collective communication。这个顺序非常合理，因为只要进入多 GPU 世界，很多训练过程其实都可以被翻译成几种固定通信原语的组合。all-reduce 用来把每张卡上的结果汇总后再广播回去；all-gather 用来把各处切片重新拼回完整对象；broadcast 用来把一份对象发给所有 rank；reduce-scatter 则把“先汇总再分发”的过程压缩成一个更节省带宽的组合。

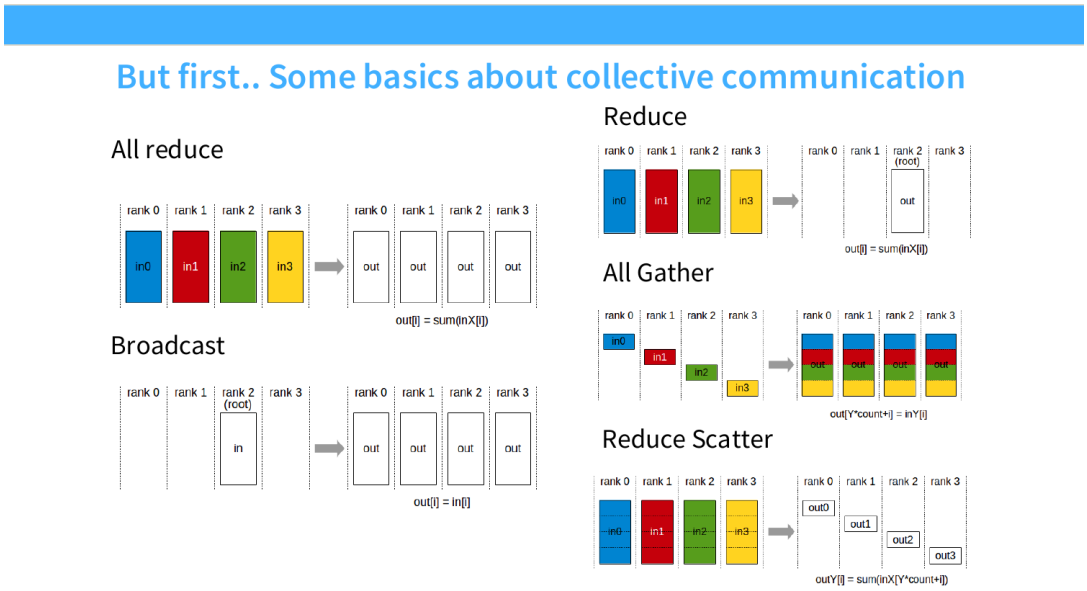

图 1: 第 7 讲先讲 collective communication，是因为后面几乎所有并行策略都可以被分解成这些通信原语。

课程还强调一个特别关键的事实：all-reduce 在带宽受限场景里，可以被理解成 reduce-scatter 再加 all-gather。这个等价关系不是细节，它直接关系到后面 ZeRO 系列为什么在通信代价上“看起来没有那么亏”。

为什么要先学 communication primitive

分布式训练看起来像很多不同工程方案，但从实现层往下看，很多差异最终都落在“哪些 tensor 在什么时候做哪种 collective”上。先懂 primitive，后面就不容易被复杂框架名词绕晕。

2.2 TPU 与 GPU 在通信上的差异为什么重要

课程顺手提到 TPU mesh 与 GPU all-to-all 网络的不同，这件事的意义在于提醒读者：并行策略从来不是只看算法，还要看底层互连是什么样。一个适合高带宽 node 内 all-to-all 的方案，未必适合跨机慢链路；反之亦然。

2.3 本章小结

collectives 是第 7 讲的语法层。只要你开始说“参数怎么同步、梯度怎么发、分片怎么拼”，本质上你已经在说这些原语。

3 Data Parallel 与 ZeRO：最自然也最容易撞墙的并行起点

3.1 Naive data parallel 的优点和硬伤

最直观的并行方案就是数据并行：每张卡拿同一模型副本，切 batch，各自做前向反向，最后同步梯度。课程把它写成最朴素的 SGD 图景，非常好理解：算力扩展不错，因为 batch 被分到多卡上；但内存扩展几乎没有，因为每张卡都得保存完整参数。

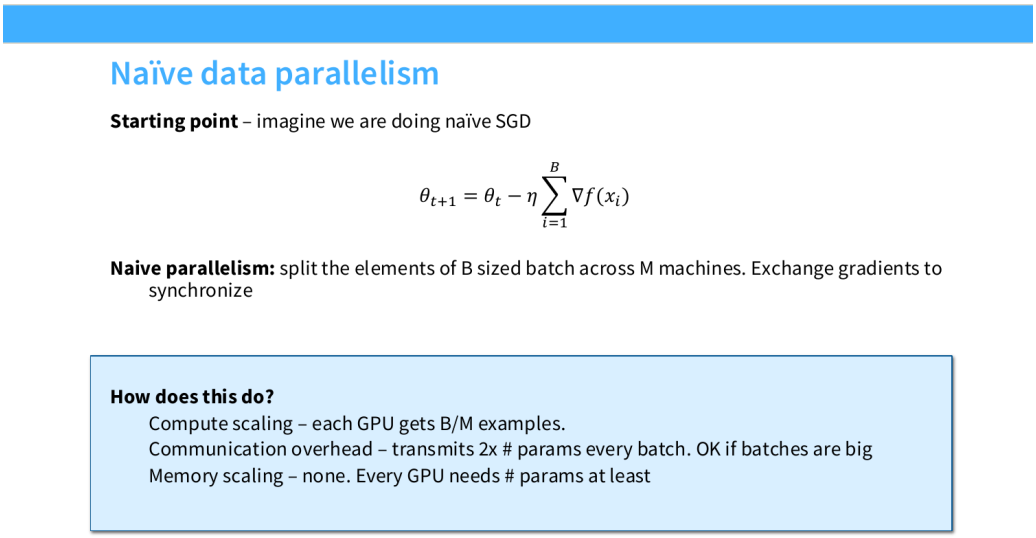

图 2: naive data parallel 的账本很直白：计算可分，但模型本体和状态几乎每卡都得完整拷贝。

这会立刻带来两个问题。第一，通信开销高，因为每个 batch 后都要同步整套梯度。第二，更致命的是显存：参数、梯度、master weights、优化器状态等对象会在每张卡上重复存在。课程用“每个参数背后可能要跟 12 到 16 bytes 甚至更多状态”这件事，再次把第 2 讲的资源账本拉进来。

naive data parallel 最大的误导

它给人一种“多加几张卡就能训更大模型”的错觉。其实如果模型副本仍然完整复制到每张卡上，那么显存上限并没有真正线性扩大。

3.2 ZeRO stage 1 / 2 / 3 在本质上分别解决什么

ZeRO 的思路可以概括成一句话：**把那些在 data parallel 里重复保存的昂贵状态切开分给不同 GPU**。

- Stage 1：先切 optimizer state。
- Stage 2：进一步连 gradients 一起切。
- Stage 3：连 parameters 本身也切，也就是 FSDP 式思路。

课程特别强调 Stage 1 的吸引力：通信代价与 naive DDP 同阶，但显存会明显改善，所以几乎像“免费午餐”。Stage 2 也很接近这个风格。到 Stage 3，通信会再涨一截，但换来的是参数本体也能真正跨卡摊开。

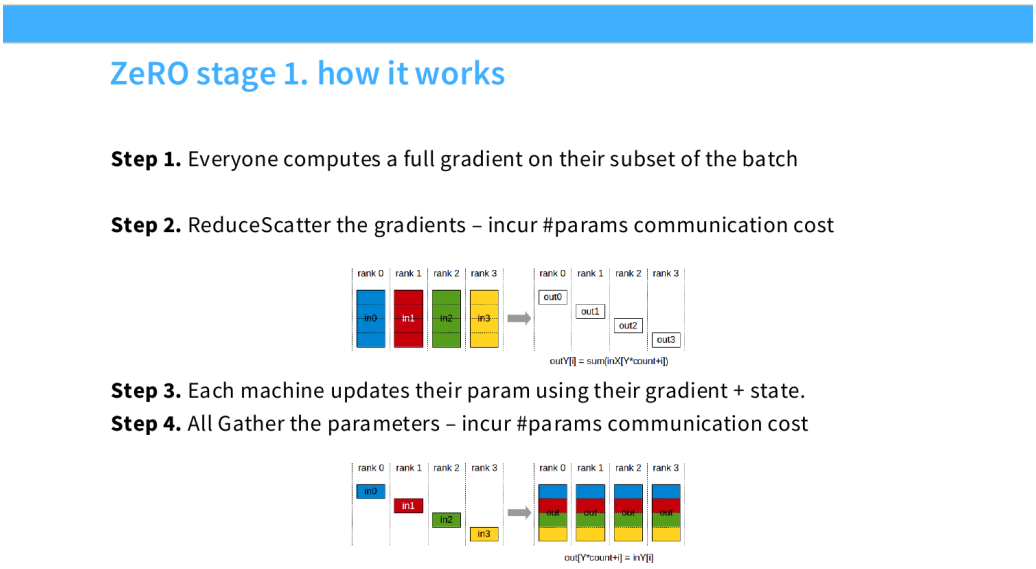

图 3: ZeRO stage 1 的典型执行流程：先 reduce-scatter 梯度，再局部更新，再 all-gather 参数。

3.3 为什么课程说 ZeRO stage 3 虽好但并不万能

Stage 3 看起来最强，因为它连参数本身都切了。但课程并没有把它描述成绝对答案，原因很实际：

- 通信更复杂，开销更高。
- activation memory 仍然不小。
- 真正运行时还要靠 incremental communication / computation overlap 才能把代价压下去。

也就是说，ZeRO stage 3 解决了“模型副本完整复制”这个问题，但没有消灭所有瓶颈。大模型训练仍然需要别的模型并行手段。

关于 ZeRO 的压缩理解

ZeRO 不是一种全新训练范式，而是 data parallel 的系统化内存削减路线：尽量保持 data parallel 的简洁，同时把最贵的重复状态一点点切开。

3.4 本章小结

data parallel 是分布式训练最自然的起点，但它会很快撞上显存和通信墙。ZeRO 的价值就在于：它让 data parallel 从“只能复制完整模型”升级成“尽量把昂贵状态切开”。

4 Model Parallel：为什么仅靠数据并行还不够

4.1 Pipeline parallel 想解决什么

当模型太大，或者即使用 ZeRO 也仍然不够时，课程引出 model parallel。最经典的第一类就是 pipeline parallel：沿深度方向切模型，把不同层段放到不同 GPU。这样做最自然的优势是内存扩展，因为每张卡只存自己那一段层。

问题也同样明显：利用率会差，因为不同 stage 之间存在等待和 bubble。课程用 micro-batch 流水线说明，只有把足够多小 batch 推进 pipeline，才能把 bubble 比例压低。

4.2 为什么 pipeline 常被放在慢链路上

课程给出一个很有工程味的判断：pipeline parallel 更适合放到跨机、较慢互连上。原因是它主要传 activations，而不是像某些别的并行方式那样反复传大块参数或频繁做 all-reduce。换句话说，它牺牲的是部分利用率，换来的是更可承受的跨机通信模式。

pipeline parallel 的工程直觉

它通常不是最优吞吐方案，但在慢网络和大模型条件下，经常是“能让系统先活下来”的方案，因为它相对节省内存，而且点对点激活传输比更激烈的全局同步容易承受。

4.3 Tensor parallel 为什么看起来更“优雅”

相比按层切，tensor parallel 是按宽度切，把矩阵乘法拆到多个 GPU 上共同完成。它的好处是没有 pipeline 那样明显的 bubble，利用率常常更好；代价则是对高带宽、低延迟互连要求更高，因此通常更适合 node 内。

课程这里的重点不只是介绍 tensor parallel 本身，而是在给读者建立一个非常重要的 mapping：

- **Pipeline parallel**：按深度切，偏内存友好、偏慢链路可用。
- **Tensor parallel**：按宽度切，偏高吞吐、偏快链路依赖。

4.4 为什么 sequence parallel 会自然冒出来

一旦你用 tensor parallel 切矩阵，很快会遇到一些“不是矩阵乘法的大层”也要跟着协调的问题，例如 layer norm、dropout、residual 等。这就是 sequence parallel 出现的动机：把部分 activation 也沿 sequence 维分片，以减少某些操作上的冗余与同步成本。

4.5 本章小结

model parallel 的意义不是取代 data parallel，而是补它的短板：当模型太大、stage 太重、activation 太多时，你需要从“切 batch”转向“切模型本体”。

5 为什么真实训练往往要把多种并行同时用上

5.1 单一并行策略很少独自解决全部问题

第 7 讲最后最重要的系统观，是：**大型训练很少只用一种并行**。原因非常简单，因为不同并行策略解决的是不同瓶颈：

- Data parallel / ZeRO 主要帮助切 batch 和切状态。
- Pipeline parallel 主要帮助切深度和省内存。
- Tensor parallel 主要帮助切宽度与提升同层并行。
- Sequence parallel 进一步减少 activation 相关开销。

所以真实训练系统常常是这些技术的组合，而不是单选题。

5.2 课程真正想教的是“哪种通信花在什么地方”

这一讲如果只把各种 parallelism 名字记下来，其实还不够。真正重要的是理解每种方案把通信预算花在了哪里：是梯度同步？是参数 all-gather？是 activation 点对点传输？还是 layer 内 all-reduce？当你能把这一层看清，后面关于 scaling 和训练大模型的讨论就会更有抓手。

并行训练里最容易犯的认知错误

不要只盯“理论线性扩展”几个字。很多并行方案纸面上都能扩展，但真正决定能不能落地的，是显存、通信模式、互连条件和利用率之间的综合取舍。

5.3 本章小结

第 7 讲真正做成的事，是把分布式训练从“黑箱框架调用”拆回成几类可理解的通信与分片结构。到这里你应该已经能看懂：为什么大模型训练几乎总是 data parallel、ZeRO、pipeline、tensor 和 sequence parallel 的混合物。

6 总结与延伸

第 7 讲最重要的结论可以压缩成四点。

第一，**数据中心已经成为新的计算单元**。单卡视角对大模型训练来说已经不够。

第二，**collective communication 是分布式训练的语法层**。不懂 all-reduce、all-gather、reduce-scatter，就很难真正理解后面的系统设计。

第三，**ZeRO 让 data parallel 从“整份复制”变成“状态尽量切开”**。它极大改善了显存效率，但仍有边界。

第四，**模型并行不是一招制胜，而是一组互补工具**。pipeline、tensor、sequence parallel 各自解决不同瓶颈，真实系统通常要组合使用。

我对第 7 讲的压缩结论

大模型并行训练的本质，不是“多卡一起跑”这么简单，而是围绕参数、梯度、优化器状态、激活与通信链路做的一整套资源重排。

下一讲继续 parallelism，但会更进一步地把这些思路推进到更复杂的系统组织方式与现实大规模训练配置里。也就是说，第 7 讲给的是语法和基本原语，第 8 讲则会更像完整系统组合学。